Universidad de San Carlos de Guatemala
Centro Universitario de Occidente "CUNOC"
División Ciencias de la Ingeniería
Laboratorio de Lenguajes Formales y de Programación
Ing. Daniel Gonzalez

# Manual Técnico
# PromptZal v3. - Compilador

William Eduardo Gómez Sánchez
Carné: 202232064

Guatemala, Quetzaltenango, 28 de octubre de 2026

## Introducción

El presente documento describe los aspectos técnicos del compilador desarrollado para el lenguaje PromptZal, como parte del Proyecto 2 del Laboratorio de Lenguajes Formales y de Programación.

PromptZal es un lenguaje diseñado para que los programadores puedan redactar prompts estructurados de forma rápida. Un programa PromptZal, escrito en un archivo con extensión .pz, describe agentes, variables y comandos de inteligencia artificial; el compilador desarrollado en este proyecto lo traduce al promptzal: el prompt final en texto, listo para copiarse y pegarse en una herramienta de inteligencia artificial.

Este proyecto parte del analizador léxico construido en el Proyecto 1, cuyo reconocimiento estaba respaldado por un autómata finito determinista (AFD) y una gramática regular, y lo completa con las fases restantes de un compilador. En esta entrega, el análisis léxico se genera con la herramienta JFlex a partir del mismo diseño de tokens del Proyecto 1, de modo que el enfoque se traslada al análisis sintáctico. Este es un analizador descendente recursivo escrito a mano, que verifica la estructura del programa según una gramática libre de contexto, con un método por cada no terminal, y que se recupera de los errores sintácticos en modo pánico para reportarlos todos en una sola ejecución.

Mientras el analizador sintáctico reconoce el programa, construye un árbol de sintaxis abstracta (AST) y, a partir de este, se genera la tabla de símbolos y se validan las reglas de significado del lenguaje (identificadores declarados antes de su uso, agentes únicos, y referencias válidas en EJECUTAR y EXPORTAR). Finalmente, el compilador recorre el árbol para generar el promptzal.

El sistema se opera mediante una interfaz gráfica desarrollada en Java Swing, con un editor, un botón para compilar, y paneles para el promptzal generado, los tokens, los errores léxicos, sintácticos y semánticos, y la tabla de símbolos. Además, genera reportes en HTML (tokens, errores, tabla de símbolos, AST y estadísticas) y la graficación del AFD y del AST mediante Graphviz.

# Requisitos del Sistema

**Requisitos mínimos para ejecutar el proyecto:**

- Sistema Operativo: Windows, Linux o macOS
- Java Development Kit (JDK): Versión 17 o superior
- Memoria RAM: 512 MB mínimo
- Espacio en disco: 50 MB mínimo
- Un archivo de texto con extensión .pz para compilar
- Graphviz (opcional): necesario únicamente para generar las imágenes del AFD y del árbol de sintaxis abstracta (AST). Debe estar instalado y disponible en el PATH del sistema. Sin Graphviz, el compilador funciona con normalidad y los reportes HTML se generan igual; el reporte del AST muestra solo su versión en texto.

**Para compilar el proyecto:**

- Apache Maven 3.6 o superior
- NetBeans IDE 17 o superior (recomendado)
- Conexión a internet en la primera compilación: Maven descarga automáticamente el plugin de JFlex (jflex-maven-plugin), por lo que no es necesario instalar JFlex por separado. El analizador léxico se genera a partir del archivo src/main/jflex/promptzal.flex en cada compilación.

**Para ejecutar el JAR:**

```
java -jar Promptzal-1.0-SNAPSHOT.jar
```

**Para compilar desde NetBeans:**

1. Abrir el proyecto en NetBeans
2. Click derecho en el proyecto
3. Seleccionar “Clean and Build”
4. El JAR se genera en la carpeta target

**Para compilar desde consola:**

1. Abrir CMD o PowerShell
2. Navegar a la carpeta raíz del proyecto: cd ruta\del\proyecto
3. Ejecutar: mvn clean package
4. El JAR se genera en la carpeta target

**Para ejecutar desde consola:**

1. Navegar a la carpeta target: cd ruta\del\proyecto\target

2. Ejecutar:

```
java -jar Promptzal-1.0-SNAPSHOT.jar
```

**Nota: los reportes HTML, el promptzal y las imágenes se escriben en la carpeta recursos, creada en el directorio desde el cual se ejecuta el programa.**

# Descripción de Tecnologías

**Lenguaje de programación:**
Java 17: lenguaje principal del proyecto, utilizado para implementar el analizador sintáctico, el árbol de sintaxis abstracta, la tabla de símbolos, el análisis semántico, el generador del promptzal, la interfaz gráfica y la generación de reportes.

**Herramientas de desarrollo:**

- NetBeans IDE 17: entorno de desarrollo utilizado para escribir, compilar y depurar el código.
- Apache Maven: herramienta de gestión y construcción del proyecto, utilizada para compilar, generar el analizador léxico con JFlex y producir el archivo JAR ejecutable.
- JFlex (mediante jflex-maven-plugin 1.9.1): generador de analizadores léxicos. A partir del archivo de especificación promptzal.flex produce en cada compilación la clase Java que reconoce los tokens del lenguaje. Se utiliza únicamente para la fase léxica; el analizador sintáctico, el AST, la semántica y la generación del promptzal son código Java propio, sin generadores de parsers.
- draw.io: herramienta utilizada para elaborar los diagramas de clases del proyecto.
- Graphviz: herramienta externa utilizada para graficar el autómata finito determinista (AFD) del diseño léxico y el árbol de sintaxis abstracta (AST) a partir de código DOT generado por el propio programa. Graphviz únicamente dibuja los grafos; no interviene en ninguna fase del análisis.
- Git: control de versiones, con el historial de desarrollo distribuido en commits por cada componente terminado.

**Librería gráfica:**
Java Swing: utilizada para construir la interfaz gráfica completa: el editor de texto, la barra de botones (abrir, guardar, compilar, guardar promptzal, reportes HTML y AFD), y un panel de resultados con pestañas para el promptzal generado, los tokens, los errores y la tabla de símbolos, mostrados mediante JTable. Cada categoría de token y cada tipo de error se muestran con un color distinto, definidos de forma centralizada en la clase Colores.

**Formato de salida:**

- HTML y CSS: utilizados para generar los cinco reportes (errores, tabla de símbolos, AST, tokens y estadísticas), con estilo visual consistente con el tema de la interfaz gráfica. El reporte del AST incluye el diagrama generado con Graphviz y una representación en texto con sangrías.
- Markdown / texto plano: formatos en que se guarda el promptzal generado (.md o .txt).

**Características técnicas:**

- Manejo de archivos: se utiliza Files.readAllBytes() para leer el contenido del archivo .pz, y FileWriter y Files.writeString() para escribir el promptzal, los reportes HTML y el código DOT.
- Colecciones: List (ArrayList) para tokens, errores y los nodos hijos del AST, ya que su cantidad no se conoce de antemano; Set para los tokens de sincronización del modo pánico; y Map (TreeMap) para contar los tokens por tipo en las estadísticas.
- Procesos externos: se utiliza ProcessBuilder para invocar a Graphviz como proceso del sistema operativo y generar las imágenes a partir del código DOT.
- Enumeraciones: TipoToken (los terminales del lenguaje), Simbolo.Tipo (agente, variable o resultado de comando) y Termino.Tipo (cadena, identificador, número o CARGAR), que evitan comparar tipos mediante cadenas de texto sueltas.
- Recursividad: el analizador sintáctico es un analizador descendente recursivo; cada no terminal de la gramática corresponde a un método, y los métodos se invocan entre sí siguiendo la estructura de las producciones.

**Conceptos de POO aplicados:**

- Encapsulamiento: los atributos de las clases son privados, con acceso mediante getters o métodos públicos específicos.
- Separación de responsabilidades (arquitectura por capas): el proyecto se organiza en paquetes según su responsabilidad: modelo, lógica de análisis, árbol de sintaxis, generadores, manejo de archivos, controlador y vista.
- Composición: el AST se construye componiendo objetos; un Programa contiene Agentes, un Agente contiene Variables y Comandos, un Comando contiene Expresiones y Conectores, y así sucesivamente hasta los Términos.
- Manejo de excepciones: se definen excepciones propias (ExcepcionArchivoNoEncontrado, ExcepcionGraphvizNoDisponible y ExcepcionSintactica). Esta última se lanza al encontrar un token inesperado y se captura en puntos concretos del analizador sintáctico, donde se aplica la recuperación en modo pánico para continuar el análisis.

## Arquitectura del Proyecto

El proyecto está organizado en paquetes según su responsabilidad, siguiendo una separación clara entre el modelo de datos, el análisis, el árbol de sintaxis, la generación de salidas, el manejo de archivos, la coordinación del flujo y la presentación:

**app** → Contiene la clase principal Promptzal, con el método main, encargada únicamente de iniciar la interfaz gráfica.

**modelo** → Contiene las clases de dominio: Token, ErrorLexico, ErrorSintactico y ErrorSemantico (uno por cada fase donde puede ocurrir un error), Simbolo (entrada de la tabla de símbolos), la enumeración TipoToken (los terminales del lenguaje) y las clases Estado, Transicion y TipoEstado, con las que se describe el AFD del diseño léxico.

**ast** → Contiene los nodos del árbol de sintaxis abstracta: Programa, Directiva, Agente, Variable, Comando, Conector, Expresion, Termino y Referencia.

**logica** → Contiene el análisis del programa: AnalizadorLexicoJFlex (clase generada por JFlex), Parser (analizador sintáctico descendente recursivo), AnalizadorSemantico, TablaSimbolos y, como respaldo del diseño léxico de la versión anterior, AnalizadorLexico y DefinicionAFD.

**generadores** → Contiene GeneradorPromptzal (prompt final a partir del AST), GeneradorReporteCompilador (los cinco reportes HTML y el diagrama del AST) y GeneradorAFD (código DOT del autómata y su imagen mediante Graphviz).

**archivos** → Contiene GestorArchivos, encargada de leer y escribir archivos desde el sistema de archivos.

**controlador** → Contiene ControladorAnalisis, que ejecuta las fases en orden (léxica, sintáctica, semántica y generación) y entrega los resultados a la interfaz.

**excepciones** → Contiene las excepciones propias ExcepcionArchivoNoEncontrado, ExcepcionGraphvizNoDisponible y ExcepcionSintactica.

**vista** → Contiene VentanaPrincipal (interfaz gráfica en Java Swing) y Colores (paleta centralizada, con un color por categoría de token y por tipo de error).

Cada fase del compilador consume el resultado de la anterior sin conocer su implementación: el analizador sintáctico recibe una lista de tokens, la semántica y los generadores reciben el árbol, y ninguno depende de la interfaz gráfica. El controlador es el único que conoce el orden de las fases, y la vista únicamente solicita la compilación y muestra los resultados, conforme al patrón MVC.

# Descripción de Clases

## Paquete app

**Promptzal** — Clase principal del proyecto. Contiene únicamente el método main, que instancia la ventana principal de la interfaz gráfica y la muestra en pantalla. Toda la lógica de negocio se delega a las demás clases; esta clase no contiene ninguna decisión de flujo.

## Paquete modelo

**Token** — Representa un componente léxico reconocido. Almacena el número correlativo del token, su lexema (el texto exacto reconocido), su tipo (por ejemplo ID o NUMERO, según los terminales de TipoToken), y la fila y columna donde comienza en el archivo original.

**TipoToken** — Enumeración con los terminales de la gramática: DIRECTIVA, AGENTE, CONTEXTO, VARIABLE, EJECUTAR, EXPORTAR, COMANDO_IA, CARGAR, CONECTOR, FLECHA, IGUAL, MAS, LLAVE_A, LLAVE_C, PAR_A, PAR_C, COMA, ID, CADENA, NUMERO y EOF. Los nombres se usan de forma consistente en el archivo JFlex, la gramática y el analizador sintáctico.

**ErrorLexico** — Representa un error léxico detectado durante el análisis. Almacena el lexema o carácter problemático, una descripción textual del error (por ejemplo “Cadena sin cerrar”), y la fila y columna donde ocurrió.

**ErrorSintactico** — Representa un error de estructura. Almacena el token encontrado, lo que se esperaba, y la fila y columna donde ocurrió.

**ErrorSemantico** — Representa un error de significado. Almacena la descripción del error y la línea donde ocurrió.

**Simbolo** — Entrada de la tabla de símbolos. Almacena el nombre, el tipo (agente, variable declarada o resultado de comando), el ámbito (el agente al que pertenece, o “global” para los agentes) y la línea de declaración. getTipoTexto() devuelve el tipo en texto legible para la interfaz y los reportes.

**Estado, Transicion y TipoEstado** — Datos con los que se describe el AFD del diseño léxico. Un Estado tiene identificador, etiqueta y tipo (tránsito, aceptación, aceptación sin token o error); un estado de error guarda además su mensaje. Una Transicion tiene origen, destino, el símbolo que consulta el motor, la etiqueta que se dibuja y si el carácter se guarda en el lexema (las comillas, por ejemplo, se consumen pero no se guardan).

**Paquete ast**

**Programa** — Raíz del árbol. Contiene las listas de directivas, agentes, ejecuciones y exportaciones. getValorDirectiva(nombre) devuelve el valor de una directiva (por ejemplo "@modelo") o null si no fue declarada.

**Directiva** — Nodo de una directiva: su nombre, el valor de la cadena que la acompaña y su línea.

**Agente** — Nodo de un agente. Contiene su nombre, su línea, su contexto, la lista de variables y la lista de comandos.

**Variable** — Nodo de una declaración "variable nombre = expresión". Almacena el nombre, la expresión asignada y la fila y columna de la declaración.

**Comando** — Nodo de un comando de IA. Almacena el tipo (PREGUNTAR, RESUMIR...), la expresión principal, la lista de conectores, la variable destino (la que sigue a la flecha) y las líneas del comando y del destino.

**Conector** — Pareja formada por la palabra del conector (SOBRE, DESDE, EN o COMO) y la expresión que lo acompaña.

**Expresion** — Lista de uno o más términos unidos con +. aTexto() los representa tal como se escribieron en el programa, y getFila() devuelve la línea de su primer término.

**Termino** — Elemento mínimo de una expresión. Su tipo (CADENA, ID, NUMERO o CARGAR) determina cómo se interpreta su valor. Para un número guarda además la unidad opcional (por ejemplo "palabras" en "100 palabras"), que se mantiene fuera de los identificadores para que no se valide como variable.

**Referencia** — Nombre usado en un EJECUTAR o un EXPORTAR, con su fila y columna, para poder reportar con exactitud un nombre inexistente.

**Paquete logica**

**AnalizadorLexicoJFlex** — Clase generada automáticamente por JFlex a partir de src/main/jflex/promptzal.flex; no se edita a mano. Define dos estados: YYINITIAL (reconocimiento normal) y COMENTARIO (consumo del cuerpo de un comentario de bloque).

- siguienteToken() — Devuelve el siguiente token del texto, con su tipo, lexema, fila y columna (desde 1), y un token EOF al terminar. Descarta espacios y comentarios, y los caracteres inválidos se registran como errores sin interrumpir el análisis.
- getErrores() — Devuelve la lista de errores léxicos acumulados.

**Parser** — Analizador sintáctico descendente recursivo escrito a mano. Recibe la lista de tokens (terminada en EOF), reconoce el programa según la gramática libre de contexto, construye el AST mientras avanza y acumula los errores sintácticos. Cada no terminal de la gramática tiene su propio método.

- actual() — Devuelve el token que se está mirando, sin consumirlo.
- es(tipo) — Indica si el token actual es de un tipo dado. Es el mecanismo con el que cada método decide qué producción usar, mirando un solo token de adelanto.
- avanzar() — Consume el token actual y pasa al siguiente. Nunca avanza más allá de EOF.
- esperar(tipo) — Exige un tipo concreto de token. Si coincide, lo consume y lo devuelve; si no, registra el error (token encontrado, token esperado, fila y columna) y lanza ExcepcionSintactica.
- reportar(esperado) — Registra un ErrorSintactico con el token actual.
- sincronizar() — Recuperación en modo pánico: descarta tokens hasta encontrar uno de sincronización (AGENTE, LLAVE_C, COMANDO_IA, VARIABLE, EJECUTAR, EXPORTAR o EOF).
- programa() — Punto de entrada. Reconoce directivas, agentes, ejecuciones y exportar, en ese orden, y reporta los tokens que sobren después del EXPORTAR. Siempre devuelve un Programa, aunque esté incompleto.
- directivas(), agentes(), masAgentes(), ejecuciones(), masEjecuciones(), exportar(), masIds() — Un método por cada no terminal del nivel superior. Los que representan "cero o más" terminan sin error si el token actual no es el que los inicia (producción vacía); los de "uno o más" (agentes, ejecuciones) exigen al menos un elemento.
- agente() — Reconoce AGENTE ID { contexto variables comandos }, creando el nodo del agente y completándolo con lo que reconocen los métodos siguientes.
- contexto(), variables(), comandos(), masComandos() — Reconocen las partes de un agente, en ese orden fijo. variables admite cero declaraciones; comandos exige al menos uno.
- comando() — Reconoce COMANDO_IA expresión conectores FLECHA ID. Junta los conectores en una lista y crea el nodo hasta conocer el destino.
- conectores() — Reconoce cero o más parejas CONECTOR expresión.
- expresion(), masTerminos(), termino(), unidad(), cargar() — Reconocen expresiones. termino() decide entre CADENA, ID, NUMERO (con su unidad opcional) o CARGAR mirando un token de adelanto, y si ninguno coincide reporta el error.
- agenteSeguro(), comandoSeguro(), ejecucionSegura() — Envuelven un solo intento en un bloque try/catch: si se lanza ExcepcionSintactica, llaman a sincronizar() y continúan con el siguiente elemento, de modo que se reporten todos los errores en una sola ejecución. En variables, directivas, contexto y exportar la recuperación se hace dentro del propio método.
- getErrores() — Devuelve la lista de errores sintácticos acumulados.

**AnalizadorSemantico** — Recorre el AST en el orden del programa, llena la tabla de símbolos y valida el significado del programa. Los errores se reportan con su descripción y línea, sin detener la generación del promptzal.

- analizar() — Recorre los agentes y, después, valida EJECUTAR y EXPORTAR, que consultan lo ya declarado.
- analizarAgente(agente) — Registra el agente en el ámbito global (reportando duplicados y omitiendo el cuerpo del duplicado), declara sus variables y analiza sus comandos.
- analizarComando(comando, ámbito) — Valida el argumento principal y los conectores, y después declara el destino como resultado de comando. Aplica las excepciones del lenguaje: en GENERAR, un identificador como primer argumento indica el tipo de contenido y no es variable; y en COMO (formato) y en EN dentro de TRADUCIR y CLASIFICAR (idioma o categorías), lo que sigue es un parámetro y no se valida como variable.
- validarExpresion(expresión, ámbito, saltarPrimero) — Exige que todo identificador usado esté declarado antes, dentro del mismo agente. La unidad de un número no se valida, porque no forma parte de los términos.
- validarEjecuciones() — EJECUTAR solo puede referirse a un agente existente.
- validarExportaciones() — EXPORTAR solo puede referirse a variables que existan en algún agente ejecutado.

**TablaSimbolos** — Almacena los símbolos declarados durante el análisis semántico.

- declarar(nombre, tipo, ámbito, línea) — Registra un símbolo; devuelve false si ya existía uno con ese nombre en ese ámbito.
- existe(nombre, ámbito) / buscar(nombre, ámbito) — Consultan si un símbolo existe en un ámbito. El ámbito es lo que impide que una variable declarada en un agente sea visible en otro.
- getSimbolos() — Devuelve la lista completa, usada por la interfaz y por el reporte.

**AnalizadorLexico** — Analizador léxico manual del Proyecto 1, conservado como respaldo del diseño del AFD. No se utiliza en la compilación del Proyecto 2, donde el análisis léxico lo realiza AnalizadorLexicoJFlex. En esta versión es un motor guiado por DefinicionAFD: no contiene un método por categoría de token, sino que consulta la definición del autómata en cada carácter.

- analizar() — Recorre el texto. Si el estado inicial tiene una transición para el carácter actual, arranca el motor; si no, reporta “Caracter no reconocido” y continúa.
- reconocer() — Motor del autómata. Parte del estado q0 y, por cada carácter, pregunta a DefinicionAFD a qué estado pasar. Al terminar, el tipo del estado final decide el resultado: aceptación genera un token, error genera un error léxico, y aceptación sin

token (comentarios y espacios) no genera nada. Un "otro" hacia aceptación o error no consume el carácter, porque pertenece al siguiente token.

- avanzar() — Mueve el puntero una posición hacia adelante y actualiza la fila y columna: si el carácter que se deja atrás es un salto de línea, incrementa la fila y reinicia la columna a 1; en cualquier otro caso, incrementa la columna.
- clasificarPalabra(palabra) — Clasifica un identificador como palabra reservada, comando de IA, conector o identificador, consultando listas en orden de prioridad.
- contiene(arreglo, valor) — Método de apoyo que determina si un arreglo de cadenas contiene un valor.
- esDirectivaValida(directiva) — Valida el nombre de una directiva contra modelo, rol y formato.

**DefinicionAFD** — Guarda el AFD completo como datos: la lista de estados y la lista de transiciones. Es la única fuente de verdad del autómata: de ahí salen tanto el código DOT como las decisiones del analizador.

- mover(estado, carácter) — Devuelve la transición que corresponde al carácter. Busca primero el símbolo exacto o su categoría (dígito, letra...) y solo si ninguno coincide cae en "otro".
- getEstado(id) — Devuelve un estado por su identificador.

**Paquete generadores**

**GeneradorPromptzal** — Genera el prompt final en texto a partir del AST, sin leer tokens ni el archivo original.

- generar(programa) — Escribe el encabezado, recorre las ejecuciones (EJECUTAR) en orden para incluir solo los agentes ejecutados, y termina con la lista de resultados de EXPORTAR. Un agente que nadie ejecuta se ignora, y un EJECUTAR a un agente inexistente se salta.
- encabezado() — Escribe el modelo, el rol y el formato, solo los que fueron declarados.
- seccionAgente() — Escribe el contexto del agente, sus variables con el origen de su valor (un literal o el archivo indicado en CARGAR) y una instrucción numerada por cada comando.
- instruccion(comando) — Traduce un comando a una oración en lenguaje natural según su tipo, buscando los conectores por palabra sin importar el orden en que se escribieron. COMO agrega al final el formato pedido, y un comando agregado en clase se redacta de forma genérica.
- limite(), texto(), termino(), fin() — Auxiliares de redacción: convierten un número con unidad en "un máximo de N palabras", unen los términos de una expresión, y agregan el punto final solo si la frase no termina ya en signo de puntuación.

**GeneradorReporteCompilador** — Genera los cinco reportes HTML, con los mismos colores que la interfaz gráfica.

- generarReporteErrores(léxicos, sintácticos, semánticos, ruta) — Tabla con todos los errores, su tipo, descripción, token encontrado y esperado (en los sintácticos), fila y columna; si no hay errores, lo indica de forma explícita.
- generarReporteSimbolos(tabla, ruta) — Tabla con los agentes y variables, su tipo, ámbito y línea de declaración.
- generarReporteAST(programa, ruta) — Página con el diagrama del árbol y su representación en texto con sangrías.
- generarReporteTokens(tokens, ruta) y generarReporteEstadisticas(...) — Lista de tokens por categoría, y conteos de tokens por tipo, errores por tipo y elementos del programa.
- generarDotAST(programa) / generarImagenAST(programa, rutaDot, rutaPng) — Producen el código DOT del árbol recorriéndolo y, con ProcessBuilder, invocan a Graphviz para generar la imagen.

**GeneradorAFD** — Representa gráficamente el AFD. Es dinámico: no contiene el DOT escrito a mano, sino que recorre los estados y transiciones de DefinicionAFD, de modo que si cambia una transición, cambia el diagrama.

- generarCodigoDOT() — Devuelve el código DOT del autómata completo.
- generarImagen(rutaDot, rutaImagen) — Escribe el DOT y, con ProcessBuilder, invoca a Graphviz. Si no está instalado, lanza ExcepcionGraphvizNoDisponible.

**Paquete archivos**

**GestorArchivos** — Clase encargada de las operaciones de lectura y escritura de archivos.

- leerArchivo(ruta) — Lee el contenido completo de un archivo y lo devuelve como cadena de texto; si el archivo no existe o no puede leerse, lanza ExcepcionArchivoNoEncontrado.
- guardarArchivo(ruta, contenido) — Escribe una cadena de texto en la ruta indicada, devolviendo un valor booleano que indica si la operación tuvo éxito.

**Paquete controlador**

**ControladorAnalisis** — Coordina el flujo completo del programa, actuando como intermediaria entre la interfaz gráfica y el resto del backend. Es la única clase que conoce el orden de las fases.

- compilar(contenido) — Reinicia los resultados y ejecuta las fases en orden: análisis léxico con JFlex, análisis sintáctico, análisis semántico y generación del promptzal. La generación se hace siempre con lo que el parser pudo armar, aunque haya errores.

- getTokens() — Devuelve los tokens de la última compilación, sin el EOF.
- getErroresLexicos() / getErroresSintacticos() / getErroresSemanticos() — Devuelven cada lista de errores.
- getTodosLosErrores() — Junta los tres tipos en un mismo formato (tipo, descripción, fila y columna) para el panel de errores.
- getPromptzal() / getTabla() / getPrograma() — Devuelven el prompt generado, la tabla de símbolos y el árbol.
- generarReportes(carpeta) — Delega en GeneradorReporteCompilador la generación de la imagen del AST y los cinco reportes HTML.
- generarAFD(carpeta) — Delega en GeneradorAFD la generación del código DOT y la imagen del autómata.
- abrirArchivo(ruta) / guardarArchivo(ruta, contenido) — Delegan en GestorArchivos las operaciones de lectura y escritura.

**Paquete excepciones**

ExcepcionArchivoNoEncontrado — Se lanza cuando un archivo no puede ser localizado o leído, incluyendo en su mensaje la ruta que se intentó abrir.

ExcepcionGraphvizNoDisponible — Se lanza cuando el sistema no logra ejecutar Graphviz, indicando que no está instalado o no se encuentra en el PATH del sistema operativo.

ExcepcionSintactica — Se lanza desde esperar() al encontrar un token inesperado, y se captura en puntos concretos del analizador sintáctico, donde se aplica la recuperación en modo pánico.

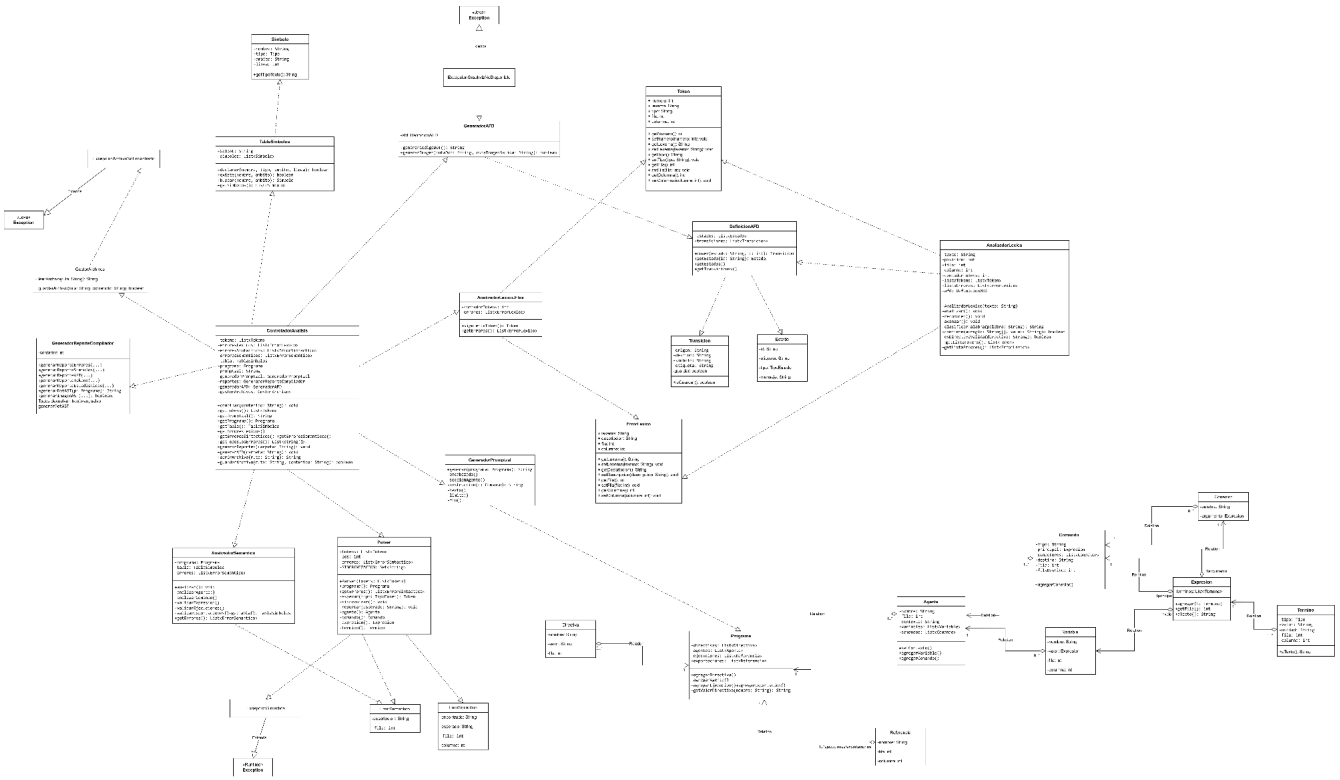
**Paquete vista**

**VentanaPrincipal** — Implementa la interfaz gráfica mediante Java Swing, con un CardLayout que alterna entre la pantalla de bienvenida y la pantalla de trabajo. Esta última tiene un editor de texto, una barra de botones (abrir, guardar, compilar, guardar promptzal, reportes HTML, AFD e inicio), una barra de estado con el resultado de cada compilación, y un panel de pestañas con el promptzal generado, los tokens, los errores y la tabla de símbolos. Delega toda la lógica en ControladorAnalisis.

- compilar() — Ejecuta la compilación y llena las pestañas; si hay errores, abre la de errores, y si no, la del promptzal.
- guardarPromptzal() — Guarda el prompt como archivo .md o .txt.
- RenderizadorColor — Clase interna que pinta cada fila de las tablas con el color de su categoría o tipo de error.

**Colores** — Centraliza la paleta de colores utilizada en toda la interfaz: fondos, acento, bordes, texto secundario y un color por categoría de token y por tipo de error. paraToken(tipo) y paraError(tipo) devuelven el color que corresponde, y los reportes HTML usan los mismos.

# Diagrama de Clases



Para visualizar el diagrama completo también se puede acceder al siguiente enlace: https://drive.google.com/file/d/1QoXpoViSqhsERFKxmMQ6SFdZEZYi6J2F/view?usp=sharing

## Tabla de Terminales

El analizador léxico, generado con JFlex, entrega al analizador sintáctico los siguientes terminales. Sus nombres coinciden con la enumeración TipoToken, el archivo promptzal.flex y los métodos del parser:

| Categoría | Ejemplos | Tipo Asignado |
|---|---|---|
| Directivas | @modelo, @rol, @formato | DIRECTIVA |
| Declaración de agente | AGENTE | AGENTE |
| Contexto del agente | contexto | CONTEXTO |
| Declaración de variable | variable | VARIABLE |
| Ejecución | EJECUTAR | EJECUTAR |
| Exportación | EXPORTAR | EXPORTAR |
| Comandos de IA | PREGUNTAR, GENERAR, RESUMIR, ANALIZAR, TRADUCIR, CLASIFICAR, EXTRAER (y cualquier comando agregado) | COMANDO_IA |
| Función del Sistema | CARGAR | CARGAR |
| Conectores | SOBRE, DESDE, EN, COMO | CONECTOR |
| Asignación de resultado | -> | FLECHA |
| Asignación | = | IGUAL |
| Concatenación | + | MAS |
| Llaves | {} | LLAVE_A, LLAVE_C |
| Paréntesis | ( ) | PAR_A, PAR_C |
| Separador | , | COMA |
| Identificadores | analista, ventas, resultado | ID |
| Literales de cadena | “texto entre comillas” | CADENA |
| Literales numéricos | 10, 100, 0.7, 3.14 | NUMERO |
| Fin del Archivo | (lo genera el analizador al terminar) | EOF |
| Comentarios | // línea, /* bloque */ | (se ignoran, no generan token) |

Nota: esta tabla describe los terminales que produce JFlex y que consume el analizador sintáctico. El analizador manual del Proyecto 1, conservado como respaldo, usa los tipos de aquella entrega.

## Categorías de token del Proyecto 1

Las siguientes categorías corresponden al analizador léxico manual del Proyecto 1 y al diseño del AFD que se presenta en esta sección. En el Proyecto 2, el análisis léxico lo genera JFlex con los terminales de la tabla anterior, que subdividen estas categorías (por ejemplo, PALABRA_RESERVADA se separa en AGENTE, CONTEXTO, VARIABLE, EJECUTAR y EXPORTAR, y ENTERO y DECIMAL se unifican en NUMERO).

| Categoría | Ejemplos | Tipo Asignado |
|---|---|---|
| Directivas | @modelo, @rol, @formato | DIRECTIVA |
| Palabras reservadas | AGENTE, contexto, variable, EJECUTAR, EXPORTAR | PALABRA_RESERVADA |
| Comandos de IA | PREGUNTAR, GENERAR, RESUMIR, ANALIZAR, TRADUCIR, CLASIFICAR, EXTRAER, CARGAR | COMANDO_IA |
| Conectores | SOBRE, DESDE, EN, COMO, -> | CONECTOR |
| Identificadores | analista, ventas, resultado | IDENTIFICADOR |
| Literales de cadena | "texto entre comillas" | CADENA |
| Literales enteros | 10, 100 | ENTERO |
| Literales decimales | 0.7, 3.14 | DECIMAL |
| Operadores | =, + | OPERADOR |
| Delimitadores | { } ( ) | DELIMITADOR |
| Comentarios | // línea, /* bloque */ | (se ignoran, no generan token) |

## Gramática libre de contexto (BNF)

La estructura del lenguaje se describe con la siguiente gramática libre de contexto. Está libre de recursividad por la izquierda y factorizada, por lo que se implementa directamente con un analizador descendente recursivo que decide con un solo token de adelanto. Los símbolos entre < > son no terminales; los de mayúsculas son los terminales de la tabla anterior; ε indica la producción vacía.

```
<programa>         -> <directivas> <agentes> <ejecuciones> <exportar> EOF

<directivas>       -> DIRECTIVA CADENA <directivas> | ε

<agentes>          -> <agente> <mas_agentes>

<mas_agentes>      -> <agente> <mas_agentes> | ε

<agente>             -> AGENTE ID LLAVE_A <contexto> <variables> <comandos>
LLAVE_C

<contexto>         -> CONTEXTO IGUAL CADENA

<variables>        -> VARIABLE ID IGUAL <expresion> <variables> | ε

<comandos>         -> <comando> <mas_comandos>

<mas_comandos>     -> <comando> <mas_comandos> | ε

<comando>          -> COMANDO_IA <expresion> <conectores> FLECHA ID

<conectores>       -> CONECTOR <expresion> <conectores> | ε

<expresion>        -> <termino> <mas_terminos>

<mas_terminos>     -> MAS <termino> <mas_terminos> | ε

<termino>          -> CADENA | ID | NUMERO <unidad> | <cargar>

<unidad>           -> ID | ε

<cargar>           -> CARGAR PAR_A CADENA PAR_C

<ejecuciones>      -> EJECUTAR ID <mas_ejecuciones>

<mas_ejecuciones> -> EJECUTAR ID <mas_ejecuciones> | ε

<exportar>         -> EXPORTAR ID <mas_ids>

<mas_ids>          -> COMA ID <mas_ids> | ε
```

**Correspondencia con el parser:** cada no terminal es un método de la clase Parser con el mismo nombre (masComandos, masTerminos, masIds...). Un terminal se traduce como una llamada a esperar(); una alternativa, como un if sobre es(); y la producción vacía, como no consumir nada. Las reglas <mas_...> expresan "uno o más" sin recursividad por la izquierda. El orden fijo del programa y del agente lo impone la secuencia de llamadas dentro de programa() y agente().

## Autómata Finito Determinista y Gramática Regular (diseño léxico)

El diseño léxico del Proyecto 1 se conserva como base del archivo promptzal.flex: JFlex implementa con una herramienta profesional el mismo reconocimiento que describe este autómata. Del estado inicial (q0) se ramifican los caminos según el primer carácter: espacio en blanco (lazo que lo descarta), símbolo suelto (operador o delimitador), guion (flecha), dígito (número), comilla (cadena), arroba (directiva), diagonal (comentario) y letra (identificador o palabra).

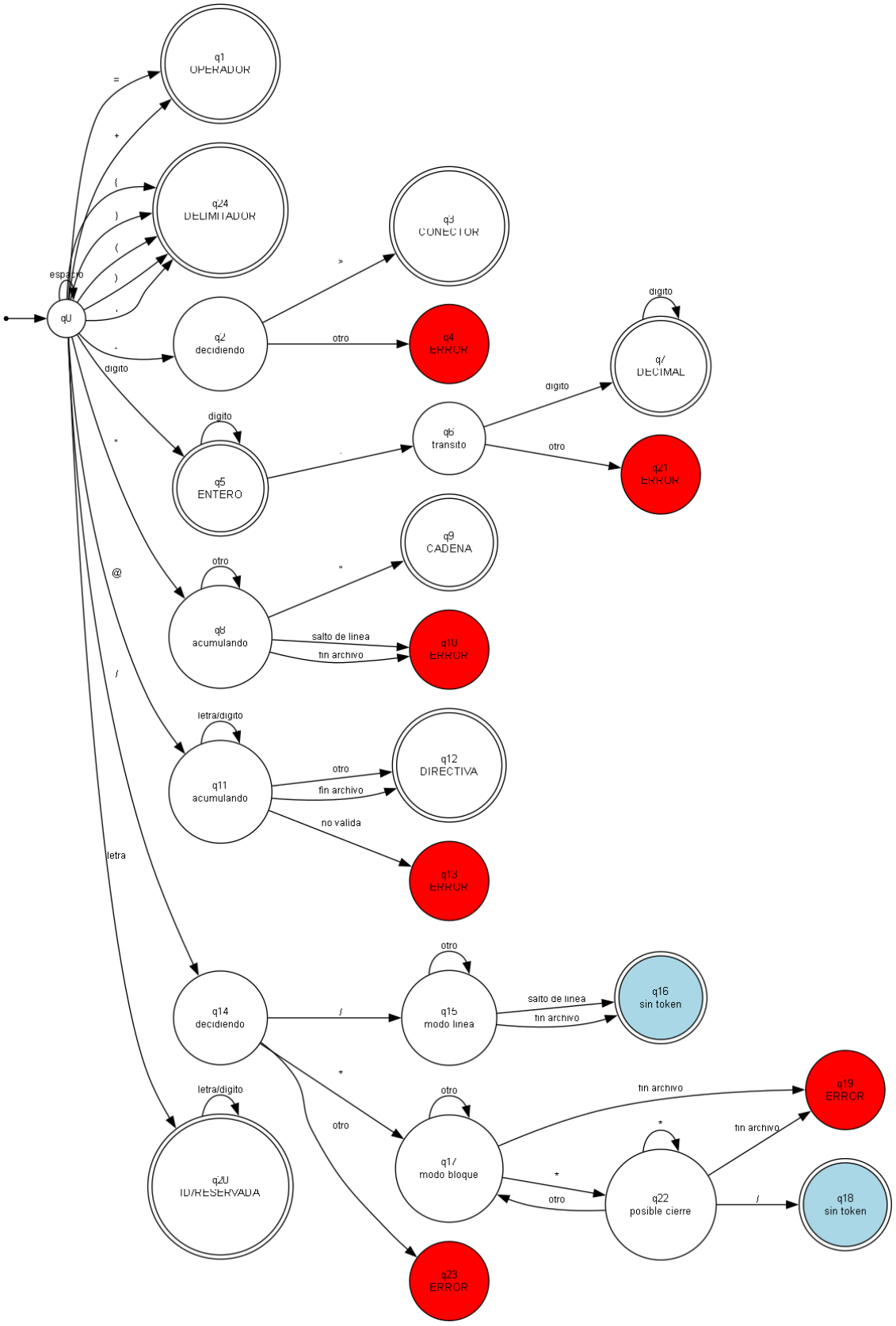
**Convención del diagrama:** círculo simple blanco, estado de tránsito; doble círculo blanco, aceptación que genera token; doble círculo celeste, aceptación sin token (comentarios); círculo rojo, estado de error.

Respecto a la primera versión, el autómata incorpora: el lazo de espacios y saltos de línea en q0; un estado de aceptación distinto para operadores (q1) y delimitadores (q24); el estado de error q21 cuando después de un punto no hay dígito (por ejemplo `12.`); el estado q22 que separa el cierre `*/` en dos transiciones (primero `*`, después `/`), con lo que `***/` cierra correctamente; y el estado de error q23 para una diagonal sola.

El autómata es dinámico: el diagrama no está escrito a mano, sino que GeneradorAFD lo genera recorriendo los estados y transiciones guardados en DefinicionAFD, la misma definición que consulta el analizador manual de respaldo. Si cambia una transición, cambia el diagrama.

La gramática regular equivalente usa l (letra o guion bajo), d (dígito), w (espacio, tabulación o salto de línea) y c (cualquier carácter permitido en esa posición). Los estados de error no aparecen, porque la gramática describe solo los lexemas válidos.

# AFD


q1
OPERADOR
q24
DELIMITADOR
q3
CONECTOR
q0
q2
decidiendo
q4
ERROR
q7
DECIMAL
q5
ENTERO
q6
transito
q21
ERROR
q9
CADENA
q8
acumulando
q10
ERROR
q11
acumulando
q12
DIRECTIVA
q13
ERROR
q14
decidiendo
q15
modo linea
q16
sin token
q20
ID/RESERVADA
q17
modo bloque
q19
ERROR
q22
posible cierre
q18
sin token
q23
ERROR
espacio
=
+
{
}
(
)
,
>
-
digito
"
@
/
letra
otro
salto de linea
fin archivo
no valida
letra/digito
*

# Gramática Regular

Identificador / palabra:

<PALABRA> → l <RESTO_PALABRA> | l

<RESTO_PALABRA> → l <RESTO_PALABRA> | d <RESTO_PALABRA> | l | d

Operador y delimitador:

<OPERADOR> → = | +

<DELIMITADOR> → { | } | ( | ) | ,

Flecha:

<FLECHA> → - <FLECHA_FIN>

<FLECHA_FIN> → >

Número:

<NUMERO> → d <ENTERO> | d

<ENTERO> → d <ENTERO> | d | . <FRAC_INICIO>

<FRAC_INICIO> → d <FRAC> | d

<FRAC> → d <FRAC> | d

Cadena (c = cualquier carácter salvo " y salto de línea):

<CADENA> → " <CONTENIDO_CADENA>

<CONTENIDO_CADENA> → c <CONTENIDO_CADENA> | "

Directiva:

```
<DIRECTIVA>            → @ <CONTENIDO_DIRECTIVA>

<CONTENIDO_DIRECTIVA>  → l <CONTENIDO_DIRECTIVA> | d
<CONTENIDO_DIRECTIVA> | l | d
```

```
Comentarios (cl = todo menos \n; cb = todo menos *; cp = todo menos
* y /):

<COMENTARIO>           → / <COMENTARIO_DECISION>

<COMENTARIO_DECISION> → / <CONTENIDO_LINEA> | * <CONTENIDO_BLOQUE>

<CONTENIDO_LINEA>      → cl <CONTENIDO_LINEA> | cl | \n

<CONTENIDO_BLOQUE>     → cb <CONTENIDO_BLOQUE> | * <POSIBLE_CIERRE>

<POSIBLE_CIERRE>       → / | * <POSIBLE_CIERRE> | cp
<CONTENIDO_BLOQUE>
```

```
Espacios:

<ESPACIO>        → w <ESPACIO> | w
```

La clasificación posterior (palabra reservada, comando, conector, validez de la directiva) se hace al aceptar el lexema, fuera de la gramática. El comentario de línea también termina al fin del archivo, que no es un carácter del alfabeto.

## Diseño del AST

El árbol de sintaxis abstracta (AST) es la representación en memoria del programa. Se construye durante el análisis sintáctico: cada método del parser devuelve el nodo correspondiente a lo que acaba de reconocer, y el método que lo invocó lo agrega a su propio nodo, de modo que el árbol se arma de abajo hacia arriba. Después del parser, ninguna fase vuelve a leer tokens ni el texto del programa: la semántica y los generadores recorren el árbol.

| **Nodo** | **Contiene** | **Regla que representa** |
|---|---|---|
| Programa | Listas de directivas, agentes, ejecuciones y exportaciones | `<programa>` |
| Directiva | Nombre, valor y línea | `DIRECTIVA CADENA` |
| Agente | Nombre, línea, contexto, variables y comandos | `<agente>` |
| Variable | Nombre, expresión, fila y columna | `VARIABLE ID IGUAL <expresion>` |
| Comando | Tipo, expresión principal, conectores, destino y líneas | `<comando>` |
| Conector | Palabra (SOBRE, DESDE, EN, COMO) y expresión | `CONECTOR <expresion>` |
| Expresión | Lista de términos | `<expresion>` |
| Término | Tipo (CADENA, ID, NUMERO, CARGAR), valor y unidad opcional | `<termino>` |
| Referencia | Nombre, fila y columna | `EJECUTAR ID, EXPORTAR ID` |

**Por qué es abstracto:** las reglas auxiliares de la gramática (`<mas_agentes>`, `<mas_comandos>`, `<mas_terminos>`, `<mas_ids>`...) existen solo para evitar la recursividad por la izquierda; en el árbol desaparecen y quedan como listas. Los tokens de puntuación (`{`, `}`, `=`, `->`, `(`, `)`) se consumen pero no se guardan. Cada nodo conserva su fila para poder reportar errores semánticos.

**Ejemplo:** `RESUMIR tendencias EN 100 palabras -> resumen` produce un Comando de tipo RESUMIR, con una Expresion principal (un Termino ID: tendencias), un

Conector EN (con una Expresion cuyo Termino es el número 100 con unidad “palabras”) y el destino “resumen”.

Si hay errores sintácticos, el árbol queda incompleto: los nodos de lo que falló no se crean, y el análisis continúa con lo siguiente.

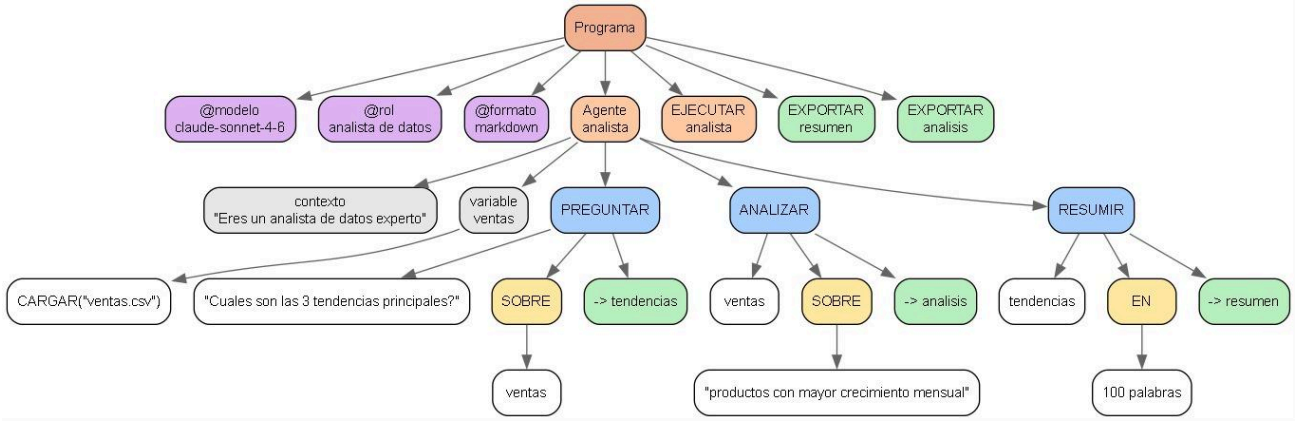



Figura: AST del programa de referencia, generado por el propio compilador con Graphviz

## Descripción del Recorrido

El compilador procesa el programa en cuatro fases, coordinadas por ControladorAnalisis.compilar(), cada una sobre el resultado de la anterior:

1. **Análisis léxico (JFlex).** AnalizadorLexicoJFlex recorre el texto y entrega un token en cada llamada a siguienteToken(), con su tipo, lexema, fila y columna desde 1. Descarta espacios y comentarios, registra los errores léxicos sin interrumpirse (carácter no reconocido, cadena sin cerrar, comentario sin cerrar, directiva no reconocida y número incompleto) y termina con el token EOF. Para decidir entre reglas, JFlex elige el lexema más largo y, a igual longitud, la regla escrita primero.
2. **Análisis sintáctico.** El Parser recibe la lista de tokens y la recorre con un solo token de adelanto. Comienza en programa() y cada método reconoce su no terminal y devuelve el nodo del árbol correspondiente, con lo que el AST se construye mientras se analiza. Ante un token inesperado, esperar() registra el error y lanza ExcepcionSintactica; la excepción se captura en un comando, variable, agente, ejecución o exportar, sincronizar() descarta tokens hasta uno de sincronización y el análisis continúa, de modo que se reportan todos los errores en una corrida.
3. **Análisis semántico.** AnalizadorSemantico recorre el árbol en el orden del programa, llena la tabla de símbolos (agentes y variables con tipo, ámbito y línea) y valida: identificadores declarados antes dentro del mismo agente, agentes sin duplicar, EJECUTAR a agentes existentes y EXPORTAR de variables existentes en agentes ejecutados, respetando las excepciones del lenguaje.
4. **Generación del promptzal.** GeneradorPromptzal recorre el árbol y escribe el encabezado con las directivas, una sección por cada agente ejecutado (contexto, variables con el origen de su valor y una instrucción por comando) y la lista de resultados de EXPORTAR.

La generación se hace siempre con lo que el parser pudo armar, aunque haya errores: los de las tres fases se muestran juntos en el panel de errores, con un color por tipo. Después de compilar, la interfaz muestra el promptzal, los tokens, los errores y la tabla de símbolos, y permite guardar el prompt y generar los reportes.

## Manejo de Errores

El compilador reporta errores en tres fases: léxica, sintáctica y semántica. Ninguna se detiene ante un error: cada una continúa hasta procesar todo el archivo y acumula sus errores en una lista propia, lo que permite reportar en una sola ejecución todos los errores del programa. Los de las tres fases se muestran juntos en el panel de errores de la interfaz y en el reporte HTML, con un color distinto por tipo (rojo para léxicos, ámbar para sintácticos y violeta para semánticos).

### Errores léxicos (JFlex)

El analizador léxico contempla cinco tipos de error:

1. Carácter no reconocido: ocurre cuando se encuentra un símbolo que no corresponde a ninguna categoría válida del lenguaje (por ejemplo `#`, `$`, `%`, o un guion o una diagonal sueltos). Se registra el carácter con su fila y columna y el análisis continúa con el siguiente. Es la recuperación más económica: se descarta un único carácter.
2. Cadena sin cerrar: ocurre cuando se encuentra una comilla de apertura pero se llega a un salto de línea o al final del archivo sin hallar la de cierre. Se registra el contenido leído, con la fila y la columna donde empezó la cadena, y el análisis continúa en la línea siguiente. Dado que el reconocimiento es determinista y sin memoria de lo ocurrido antes, cualquier comilla suelta se interpreta siempre como una apertura; no es posible distinguir si se trataba de una cadena sin abrir o sin cerrar.
3. Comentario de bloque sin cerrar: ocurre cuando se encuentra la apertura `/*` pero se llega al final del archivo sin hallar `*/`. A diferencia de los casos anteriores, no existe ningún carácter que sirva como límite no ambiguo, porque un comentario de bloque válido puede extenderse por varias líneas y contener cualquier carácter. Por eso la única recuperación posible es reportar un único error, ubicado en la apertura, y descartar el resto del archivo como parte del comentario.
4. Directiva no reconocida: ocurre cuando después de `@` viene un nombre distinto de `modelo`, `rol` y `formato` (por ejemplo `@xyz`), o no viene ninguno. Se registra la directiva completa y el análisis continúa.
5. Número incompleto: ocurre cuando después de un punto decimal no hay un dígito (por ejemplo `12.`). Corresponde al estado de error q21 del autómata. Se registra el lexema acumulado (`12.`) y el análisis continúa; el reporte lo muestra con la descripción “Caracter no reconocido”.

**Errores sintácticos (analizador descendente recursivo)**

Se reportan cuando el token que llega no corresponde a lo que la gramática esperaba. Cada error registra el token encontrado, el token esperado, la fila y la columna; por ejemplo: "Token encontrado: ID (resultado). Se esperaba: FLECHA".

La recuperación es en modo pánico:

1. El método `esperar()` detecta el token inesperado, registra el error y lanza ExcepcionSintactica.
2. La excepción se captura en un punto concreto: un comando, una variable, un agente, una ejecución, las directivas, el contexto o el exportar.
3. `sincronizar()` descarta tokens hasta encontrar uno de sincronización: AGENTE, LLAVE_C, COMANDO_IA, VARIABLE, EJECUTAR, EXPORTAR o EOF.
4. El análisis continúa desde ahí, de modo que un archivo con varios errores los reporta todos.

El ciclo siempre termina: cada elemento que se reintenta (un comando, una variable, un agente) consume al menos su primer token antes de poder fallar, así que cada vuelta avanza. Si el programa tiene tokens de sobra después del EXPORTAR, se reporta un único error esperando EOF. Los elementos donde se produjo el error no llegan al árbol, que queda incompleto pero válido.

**Errores semánticos (análisis del árbol)**

Se reportan con su descripción y la línea donde ocurren, y no detienen la generación del promptzal. Las validaciones son:

- Un identificador usado en una expresión debe estar declarado antes, dentro del mismo agente.
- No puede haber dos agentes con el mismo nombre; el cuerpo del duplicado no se analiza, para no mezclar sus símbolos con los del original.
- EJECUTAR solo puede referirse a un agente que exista.
- EXPORTAR solo puede referirse a variables que existan en algún agente ejecutado.

Excepciones: en GENERAR, un identificador como primer argumento indica el tipo de contenido a generar (`GENERAR codigo`) y no es variable; la palabra que sigue a un número (por ejemplo `palabras` en `100 palabras`) describe el número y no se valida; y los parámetros de COMO (formato) y de EN en TRADUCIR y CLASIFICAR (idioma o categorías) tampoco se validan como variables.

**Ejemplos**

| **Fase** | **Programa** | **Error Reportado** |
|---|---|---|
| Léxico | # | Caracter no reconocido (‘#’), fila 2, columna 1 |
| Sintáctico | PREGUNTAR "p" SOBRE v resultado | Token encontrado: ID (resultado). Se esperaba: FLECHA, fila 4, columna 27 |
| Semántico | PREGUNTAR "p" SOBRE zzz -> r | Identificador no declarado: ‘zzz’ en el agente ‘a’, línea 3 |
| Semántico | Dos agentes llamados `a` | Agente duplicado: ya existe un agente llamado ‘a’, línea 5 |
| Semántico | `EJECUTAR b` sin agente `b` | EJECUTAR hace referencia a un agente que no existe: ‘b’, línea 6 |

Estos ejemplos son los de las pruebas del compilador.

# Pseudocódigo del analizador léxico manual (Proyecto 1)

Los siguientes métodos corresponden a la primera versión del analizador, conservada como respaldo del diseño del AFD. En el Proyecto 2 el análisis léxico lo realiza JFlex.

### Método espiar()

```
espiar()
   Si posicion + 1 es menor que la longitud del texto entonces
      retornar el caracter en la posicion + 1
   Si no
      retornar caracter nulo
Fin espiar
```

### Método avanzar()

```
avanzar()
   Si el caracter en la posicion actual es salto de linea entonces
      incrementar fila
      columna = 1
   Si no
      incrementar columna
   incrementar posicion
Fin avanzar
```

### Método leerCadena()

```
leerCadena()
   guardar fila y columna de inicio
   avanzar()  // se salta la comilla de apertura
   cerrada = falso
   Mientras haya texto por leer hacer
      Si el caracter actual es comilla entonces
         avanzar()  // se salta la comilla de cierre
         cerrada = verdadero
         salir del ciclo
      Si el caracter actual es salto de linea entonces
         salir del ciclo
      acumular el caracter actual
      avanzar()
   FinMientras
   Si cerrada es verdadero entonces
      crear Token de tipo CADENA con lo acumulado
   Si no
      crear ErrorLexico "Cadena sin cerrar"
Fin leerCadena
```

**Método leerNumero()**

```
leerNumero()
   guardar fila y columna de inicio
   esDecimal = falso
   Mientras haya texto por leer hacer
      Si el caracter actual es digito entonces
         acumular el caracter, avanzar()
      Si no, si el caracter actual es punto Y no esDecimal
            Y el siguiente caracter (espiado) es digito entonces
         esDecimal = verdadero
         acumular el punto, avanzar()
      Si no
         salir del ciclo
   FinMientras
   Si esDecimal entonces
      crear Token de tipo DECIMAL
   Si no
      crear Token de tipo ENTERO
Fin leerNumero
```

**Método analizar()**

```
analizar()
   Mientras haya texto por leer hacer
      caracter_actual = obtener caracter en posicion actual
      Si es espacio o salto de linea entonces
         avanzar()
      Si no, si es letra o guion bajo entonces
         leerPalabra()
      Si no, si es arroba entonces
         leerDirectiva()
      Si no, si es comilla entonces
         leerCadena()
      Si no, si es digito entonces
         leerNumero()
      Si no, si es diagonal entonces
         leerComentario()
      Si no, si es guion entonces
         leerConectorFlecha()
      Si no, si es simbolo suelto valido entonces
         leerSimboloSuelto()
      Si no
         crear ErrorLexico "Caracter no reconocido"
         avanzar()
   FinMientras
```

Fin analizar

### Método leerComentario()

```
leerComentario()
   avanzar()  // se salta la primera diagonal
   siguiente = caracter actual

   Si siguiente es diagonal entonces
      // comentario de linea
      avanzar()
      Mientras haya texto y el caracter actual no sea salto de linea hacer
         avanzar()
      FinMientras
   Si no, si siguiente es asterisco entonces
      // comentario de bloque
      avanzar()
      cerrado = falso
      Mientras haya texto por leer hacer
         Si el caracter actual es asterisco Y el siguiente (espiado) es diagonal entonces
            avanzar() // salta el asterisco
            avanzar() // salta la diagonal
            cerrado = verdadero
            salir del ciclo
         avanzar()
      FinMientras
      Si cerrado es falso entonces
         crear ErrorLexico "Comentario de bloque sin cerrar"
Fin leerComentario
```

### Método leerConectorFlecha

```
leerConectorFlecha()
   guardar fila y columna de inicio
   Si el siguiente caracter (espiado) es mayor-que entonces
      avanzar() // salta el guion
      avanzar() // salta el mayor-que
      crear Token de tipo CONECTOR con lexema "->"
   Si no
      crear ErrorLexico "Caracter no reconocido" con lexema "-"
      avanzar()
Fin leerConectorFlecha
```

**Método generarCodigoDOT() (estructura resumida)**

```
generarCodigoDOT()
   iniciar cadena DOT con encabezado del grafo (rankdir, nodo inicial)
   Para cada una de las siete ramas del AFD (simbolo suelto, flecha,
   numero, cadena, directiva, comentario) hacer
      agregar la definicion de sus estados (con su forma y color
      segun sea transito, aceptacion con token, aceptacion sin
      token, o error)
      agregar las transiciones correspondientes con su etiqueta
   FinPara
   cerrar la cadena DOT
   retornar la cadena completa
Fin generarCodigoDOT
```

## Pseudocódigo de métodos del Proyecto 2

**Método esperar()**

```
esperar(tipo)
   Si el token actual es del tipo pedido entonces
      retornar avanzar()
   Si no
      reportar(tipo)          // registra token encontrado, esperado, fila y columna
      lanzar ExcepcionSintactica
Fin esperar
```

**Método sincronizar() (modo pánico)**

```
sincronizar()
   Mientras el tipo del token actual no esté en
         {AGENTE, LLAVE_C, COMANDO_IA, VARIABLE, EJECUTAR, EXPORTAR, EOF}
hacer
      pos = pos + 1            // se descarta el token
   FinMientras
Fin sincronizar
```

**Método comando() (un no terminal que construye un nodo)**

```
comando()                    // <comando> -> COMANDO_IA <expresion> <conectores>
FLECHA ID
   inicio = esperar(COMANDO_IA)
   principal = expresion()
   lista = lista vacía de conectores
   conectores(lista)
   esperar(FLECHA)
   destino = esperar(ID)
   crear Comando con el tipo de inicio, principal y destino
   agregar al Comando cada conector de lista
   retornar el Comando
Fin comando
```

**Método conectores() (producción vacía)**

```
conectores(lista)            // <conectores> -> CONECTOR <expresion> <conectores> | ε
   Si el token actual es CONECTOR entonces
      palabra = avanzar().lexema
      argumento = expresion()
      agregar Conector(palabra, argumento) a lista
      conectores(lista)
   // si no, es ε: no consume nada y termina sin error
Fin conectores
```

**Método termino() (decide con un token de adelanto)**

```
termino()
   Si el token actual es CADENA entonces
      retornar Termino de tipo CADENA
   Si no, si es ID entonces
      retornar Termino de tipo ID
   Si no, si es NUMERO entonces
      avanzar()
      unidad = unidad()        // ID opcional: "100 palabras"
      retornar Termino de tipo NUMERO con su unidad
   Si no, si es CARGAR entonces
      retornar cargar()
   Si no
      reportar("CADENA, ID, NUMERO o CARGAR")
      lanzar ExcepcionSintactica
Fin termino
```

**Método variables() (recuperación dentro del método)**

```
variables(agente)                // <variables> -> VARIABLE ID IGUAL <expresion>
<variables> | ε
   Si el token actual es VARIABLE entonces
      Intentar
         avanzar()
         nombre = esperar(ID)
         esperar(IGUAL)
         valor = expresion()
         agregar Variable(nombre, valor) al agente
      Si ocurre ExcepcionSintactica entonces
         sincronizar()
      FinIntentar
      variables(agente)
Fin variables
```

**Método agente()**

```
agente()
   esperar(AGENTE)
   nombre = esperar(ID)
   esperar(LLAVE_A)
   crear Agente con nombre
   Intentar
      asignar contexto() al agente
   Si ocurre ExcepcionSintactica entonces
      sincronizar()
   variables(agente)
   comandos(agente)
   Intentar
      esperar(LLAVE_C)
   Si ocurre ExcepcionSintactica entonces
      sincronizar()
      Si el token actual es LLAVE_C entonces avanzar()
   retornar el Agente
Fin agente
```

**Método programa() (punto de entrada)**

```
programa()
   crear Programa
   directivas(programa)
   agentes(programa)
   ejecuciones(programa)
   exportar(programa)
   Si el token actual no es EOF entonces
      reportar("EOF")          // quedaron tokens sueltos después del EXPORTAR
   retornar el Programa          // siempre se devuelve, aunque esté incompleto
Fin programa
```

**Método analizarAgente() (semántica)**

```
analizarAgente(agente)
   esNuevo = declarar(agente.nombre, AGENTE, ámbito global, agente.línea)
   Si no esNuevo entonces
      error "Agente duplicado"
      retornar                  // no se analiza su cuerpo
   Para cada variable del agente hacer
      validarExpresion(variable.valor, agente.nombre, no saltar)
      declarar(variable.nombre, VARIABLE, agente.nombre, variable.fila)
   Para cada comando del agente hacer
      analizarComando(comando, agente.nombre)
Fin analizarAgente
```

**Método analizarComando()**

```
analizarComando(comando, ámbito)
   saltarPrimero = (comando.tipo es GENERAR) Y (su primer término es un ID)
   validarExpresion(comando.principal, ámbito, saltarPrimero)
   Para cada conector del comando hacer
      esParámetro = (conector es COMO) O
                    (conector es EN Y comando es TRADUCIR o CLASIFICAR)
      Si no esParámetro entonces
         validarExpresion(conector.argumento, ámbito, no saltar)
   declarar(comando.destino, RESULTADO, ámbito, comando.filaDestino)
Fin analizarComando
```

**Método validarExpresion()**

```
validarExpresion(expresión, ámbito, saltarPrimero)
   Para cada término de la expresión, con su posición i, hacer
      Si el término no es un ID entonces continuar
      Si i es 0 Y saltarPrimero entonces continuar
      Si no existe(término.valor, ámbito) entonces
         error "Identificador no declarado" con la línea del término
Fin validarExpresion
```

**Método validarExportaciones()**

```
validarExportaciones()
   Para cada referencia de EXPORTAR hacer
      encontrada = falso
      Para cada referencia de EJECUTAR hacer
         Si existe(referencia.nombre, ejecución.nombre) entonces
            encontrada = verdadero
      Si no encontrada entonces
         error "EXPORTAR hace referencia a una variable inexistente"
Fin validarExportaciones
```

**Método generar() (GeneradorPromptzal)**

```
generar(programa)
   escribir el título del prompt
   escribir el encabezado: @modelo, @rol y @formato (solo los declarados)
   Para cada referencia de EJECUTAR, en orden, hacer
      agente = buscar el agente con ese nombre
      Si agente existe entonces
         escribir contexto, variables con su origen e instrucciones numeradas
         // una instrucción por comando, según su tipo (PREGUNTAR, RESUMIR...)
   escribir "Resultado solicitado" con cada referencia de EXPORTAR
   retornar el texto completo
Fin generar
```

**Método reconocer() (motor del AFD de respaldo)**

```
reconocer()
   estado = q0; lexema = vacío
   Repetir
      c = carácter actual (o fin de archivo)
      transición = DefinicionAFD.mover(estado, c)
      Si no hay transición entonces salir
      destino = estado destino de la transición
      consume = c no es fin de archivo
                Y destino no es de error
                Y no es (símbolo "otro" hacia un estado que no es de tránsito)
      Si consume entonces
         Si la transición guarda el carácter entonces agregar c al lexema
         avanzar()
      Si el destino es q0 entonces reiniciar lexema y posición de inicio
      estado = destino
   FinRepetir
   Según el tipo del estado final:
      aceptación → crear Token (clasificando palabras reservadas y directivas)
      error → crear ErrorLexico con el mensaje del estado
      aceptación sin token → no hacer nada
Fin reconocer
```

## Conclusión

El desarrollo del Proyecto 2 permitió completar el compilador de PromptZal a partir del analizador léxico construido en el Proyecto 1. Si en aquella entrega se formalizó el reconocimiento de cada pieza del lenguaje, en esta se logró que el sistema también verifique que esas piezas estén ordenadas correctamente, interprete su significado y produzca el resultado para el cual el lenguaje fue diseñado: el promptzal, un prompt en texto listo para usarse en una herramienta de inteligencia artificial.

El diseño del Proyecto 1 no se descartó, sino que se aprovechó: el autómata finito determinista y la gramática regular sirvieron de base para escribir el archivo de especificación de JFlex, que genera el analizador léxico con los veinte terminales que consume el analizador sintáctico. Esto permitió que el esfuerzo se concentrara en la fase nueva, y mostró la diferencia entre construir un reconocedor a mano y delegarlo a una herramienta profesional que aplica reglas como la coincidencia más larga.

El analizador sintáctico, escrito a mano como un analizador descendente recursivo, hizo evidente la relación directa entre una gramática y su código: cada no terminal es un método, cada terminal una llamada a esperar() y cada alternativa una decisión con un solo token de adelanto. Esa relación solo es posible porque la gramática está libre de recursividad por la izquierda y factorizada. La recuperación en modo pánico, basada en una excepción propia y en un conjunto de tokens de sincronización, permite reportar todos los errores sintácticos de un archivo en una sola ejecución, en lugar de detenerse en el primero.

La construcción del árbol de sintaxis abstracta durante el análisis sintáctico permitió separar las fases: después del parser, ninguna etapa vuelve a leer tokens ni texto, sino que el análisis semántico y los generadores recorren el mismo árbol y cada uno obtiene de él algo distinto. El análisis semántico, apoyado en una tabla de símbolos con tipo, ámbito y línea, validó las reglas de significado del lenguaje, incluidas las excepciones propias de GENERAR, de la unidad de los números y de los parámetros de COMO y EN, y las reportó sin impedir la generación del promptzal. Este diseño dejó además un compilador tolerante a errores: ante un programa con fallos, produce igualmente lo que pudo reconocer y muestra los errores de las tres fases juntos.

La interfaz gráfica en Java Swing, con un editor, un botón de compilación y paneles para el promptzal, los tokens, los errores y la tabla de símbolos, junto con los cinco reportes HTML y la graficación del autómata y del árbol mediante Graphviz, transformó el compilador en una herramienta operable sin conocimientos técnicos. Como limitación conocida, el generador escribe el nombre de una variable y no su valor cuando esta se usa como parámetro (por ejemplo, `EN limite`), lo que no afecta la corrección del prompt pero podría mejorarse en una versión futura.

Como resultado final se obtuvo un compilador funcional que traduce un programa PromptZal, desde su texto fuente hasta el prompt final, con una arquitectura por capas en la que cada fase conoce únicamente el resultado de la anterior.